

PROGRAMA DE ESTUDIO

Administración de Centros de Cómputo

MÓDULO

N° 3: Gestión de Plataformas y Servicios de TI

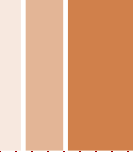
UNIDAD DIDÁCTICA

Administración de Plataformas de TI



INSTITUTO DE
EDUCACIÓN SUPERIOR
TECNOLÓGICO PÚBLICO
"PAIJÁN"

- Llegar puntual y registrar la asistencia.
- Mantener el celular en silencio durante la práctica.
- Participar activamente y respetar la opinión de los compañeros.
- Cuidar los equipos y el orden del laboratorio.
- Guardar y entregar las evidencias de cada práctica.



Semana 08

Fecha: 05/10/2026

Docente: Ing. Carlos Abilio, Angulo Zegarra

Responde en voz alta:

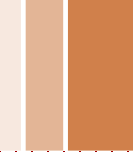
¿Qué pasa si dos administradores editan el mismo archivo de configuración a la vez?

¿Cómo sabes qué cambió en un servidor, quién lo cambió y cuándo?

¿Dónde guardas tus scripts para no perderlos si falla tu laptop?

[Material de apoyo: Aprende Git y GitHub desde cero \(freeCodeCamp Español, incluye curso en YouTube\).](#)





Situación: El script de backup funcionaba el viernes y falló el lunes. Dos administradores modificaron el mismo archivo y nadie sabe qué versión es la correcta ni quién la cambió.

Conflicto: ¿Cómo recuperar la versión que funcionaba y trabajar en equipo sin pisarse los cambios?

Al concluir la sesión los estudiantes gestionan código y documentación de infraestructura mediante Git y GitHub: crean un repositorio, registran commits, trabajan con ramas y documentan en Markdown, manteniendo orden, responsabilidad y buenas prácticas colaborativas.



Control de versiones: registra el historial de cambios de los archivos, quién los hizo y cuándo.

Git vs GitHub: Git funciona en tu equipo; GitHub es la plataforma en la nube que aloja los repositorios.

Flujo básico: git add (preparar) → git commit (guardar local) → git push (subir a GitHub).

Ramas: líneas de trabajo paralelas para probar cambios sin romper la versión estable (main).

DevOps en infraestructura: scripts, archivos YAML y configuraciones se versionan y se documentan en Markdown (README.md).

Antes de empezar, configura tu identidad con `git config --global user.name` y `user.email`.

```
● ● ● estudiante@debian: ~/infra-lab

$ mkdir infra-lab && cd infra-lab && git init -b main
Initialized empty Git repository in /home/estudiante/infra-lab/.git/
$ echo "# Infra Lab" > README.md
$ git add README.md
$ git commit -m "Primer commit: README"
[main (root-commit) 3f2a9c1] Primer commit: README
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
$ git log --oneline
3f2a9c1 (HEAD -> main) Primer commit: README
```



Probar un cambio en una rama aparte y fusionarlo a main cuando funciona.

```
estudiante@debian: ~/infra-lab

$ git switch -c feature/nginx-conf
Switched to a new branch 'feature/nginx-conf'

$ echo "worker_processes 2;" > nginx.conf

$ git add nginx.conf && git commit -m "Agrega nginx.conf"
[feature/nginx-conf a1b2c3d] Agrega nginx.conf
1 file changed, 1 insertion(+)

$ git switch main && git merge feature/nginx-conf
Updating 3f2a9c1..a1b2c3d
Fast-forward
 nginx.conf | 1 +
1 file changed, 1 insertion(+)
```



PRÁCTICA 3: SUBIR A GITHUB

Crea un repositorio vacío en GitHub y conéctalo por SSH a tu repositorio local.

```
● ● ● estudiante@debian: ~/infra-lab

$ git remote add origin git@github.com:estudiante/infra-lab.git
$ git push -u origin main
To github.com:estudiante/infra-lab.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
$ git status -sb
## main...origin/main
```

Reto: construir el repositorio de infraestructura del equipo (2-3 integrantes, 60 minutos).

Crear el repositorio infra-equipo en GitHub y clonarlo en la máquina Debian.

Agregar el docker-compose.yml de la semana 7 y un README.md en Markdown (descripción, requisitos y cómo desplegar).

Cada integrante trabaja en su propia rama y hace al menos 2 commits con mensajes claros.

Fusionar las ramas en main y subir todo con git push.

Evidencia: enlace al repositorio y captura de git log --oneline --graph.

CRITERIO	DESCRIPTOR	PUNTAJE
Conceptual	Explica la diferencia entre Git y GitHub y el flujo add, commit y push.	4
Procedimental	Crea el repositorio, registra commits, trabaja con ramas, sube a GitHub y documenta en README.md.	8
Actitudinal	Mantiene orden, responsabilidad y buenas prácticas colaborativas; entrega sus evidencias.	8
TOTAL		20



Git guarda el historial en tu equipo; GitHub lo respalda y permite colaborar.

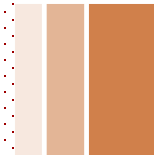
Un buen commit es pequeño y lleva un mensaje claro.

Las ramas permiten experimentar sin afectar la versión estable (main).

Versiona scripts, YAML y configuraciones, documéntalos en README.md y nunca subas contraseñas ni claves privadas.

- ¿Qué diferencia hay entre git commit y git push, y qué pasaría si olvidas hacer push?
- ¿Por qué conviene trabajar en una rama y no directamente en main?
- ¿Qué archivos de tu infraestructura nunca deberías subir a un repositorio y cómo lo evitas?

Chacon, S. y Straub, B. (2014). Pro Git (2.ª ed.). Apress. <https://git-scm.com/book/es/v2>
GitHub. (s. f.). Documentación de GitHub. <https://docs.github.com/es>
freeCodeCamp Español. (s. f.). Aprende Git y GitHub: curso desde cero.
<https://www.freecodecamp.org/espanol/news/aprende-git-y-github-curso-desde-cero/>



*Muchas
Gracias!*